Bài thực hành 06

Kiểm chứng Mô hình Giới hạn (Bounded Model Checking) & Mô hình Bộ nhớ

Hệ thống SecureSys – Module Inventory (Truy cập mảng)

Học phần CSE703093 – An toàn phần mềm
Phenikaa University

Ngày 8 tháng 7 năm 2026

Tóm tắt nội dung

Bài thực hành xây dựng một động cơ **Bounded Model Checking (BMC)** tự viết bằng Z3 để kiểm tra an toàn truy cập mảng: mở cuộn (unwind) vòng lặp k bước, mã hóa mỗi bước thành công thức SMT, kiểm tra tồn tại vi phạm khẳng định an toàn. Kết quả được đối chiếu với hành vi runtime thực tế qua AddressSanitizer.

Mục lục

1 Kiến thức nền tảng **1**

2 Bốn chương trình được kiểm chứng **2**

3 Đối chiếu với runtime (C + ASan) **2**

4 Kết quả kiểm thử tự động **2**

5 Tài liệu tham khảo **3**

1 Kiến thức nền tảng

Bounded Model Checking mở cuộn vòng lặp tối đa k bước, tại mỗi bước sinh điều kiện đường đi (path condition) và khẳng định an toàn (ở đây: `0 <= index < array_size`), rồi hỏi SMT solver: *tồn tại phép gán biến nào làm khẳng định bị vi phạm hay không?*

- SAT $\Rightarrow$ tìm thấy lỗi (UNSAFE), kèm phản ví dụ cụ thể.
- UNSAT (toàn bộ k bước) $\Rightarrow$ **SAFE up to bound** k – *không* phải chứng minh an toàn tuyệt đối, chỉ an toàn trong phạm vi unwind đã cho. Đây là giới hạn nền tảng của BMC.

2 Bốn chương trình được kiểm chứng

ID	Mô tả	Kết quả
BMC-01	Duyệt đúng `for (i=0; i<N; i++)`	SAFE
BMC-02	Off-by-one: `for (i=0; i<=N; i++)`	**UNSAFE** tại i=10
BMC-03	`catalog[base_index + i]`, `base_index` không kiểm tra	**UNSAFE**
BMC-04	Như BMC-03 nhưng có tiền điều kiện chặn `base_index`	SAFE

```python
for i in range(unwind_bound):
    solver = Solver()
    if symbolic_base:
        base_index = Int("base_index")
        solver.add(base_index >= 0, base_index < array_size)
    index_expr = base_index + i
    assertion_safe = And(index_expr >= 0, index_expr < array_size)
    solver.add(Not(assertion_safe))
    if solver.check() == sat:
        return UNSAFE_with_witness(...)
```

Listing 1: Vòng lặp mở cuộn BMC (trích bmc_engine.py)

3 Đối chiếu với runtime (C + ASan)

Nhân chứng (witness) mà BMC tìm được cho BMC-03 (`base_index=9`) được dùng trực tiếp làm tham số dòng lệnh cho `c/array_access.c symbolic_unsafe 9` – chương trình **crash ngay lập tức** với báo cáo `heap-buffer-overflow` từ ASan, xác nhận kết quả kiểm chứng tĩnh khớp với hành vi runtime thực tế.

4 Kết quả kiểm thử tự động

`tests/test_bmc.py` thực hiện 5 test, bao gồm việc xác nhận giá trị nhân chứng BMC-03 thực sự làm chương trình C sụp đổ. **Kết quả: 5/5 test PASS.**

Mục tiêu học tạp

- Hiểu nguyên lý Bounded Model Checking (mở cuộn + mã hóa SMT).
- Phân biệt ”SAFE up to bound k” với ”an toàn tuyệt đối”.
- Tự viết động cơ BMC đơn giản bằng Z3.
- Đối chiếu kết quả kiểm chứng tĩnh với hành vi runtime (ASan).

Yêu cạu hoàn thiện

1. Bảng dự đoán tay đối chiếu kết quả công cụ.
2. Giải thích thí nghiệm về giới hạn nền tảng của BMC (giảm unwind_bound).
3. Chương trình BMC-05 tự thêm (đếm ngược mảng) kèm giải thích.

Yêu cạu kẹt quạ

`tests/test_bmc.py` toàn bộ 5 test PASS; giá trị `base_index` trong nhân chứng BMC-03 thực sự làm chương trình C crash khi chạy với ASan.

5 Tài liệu tham khảo

- Đề cương CSE703093 – An toàn phần mềm, Chương 3 (3.5, 3.6, 3.7).
- E. Clarke, D. Kroening, F. Lerda, *A Tool for Checking ANSI-C Programs*, TACAS 2004 (nền tảng CBMC).